

AI-LAB ASSIGNMENT- 7.2

NAME: YEROJU ANJALI

HT.NO: 2303A51416

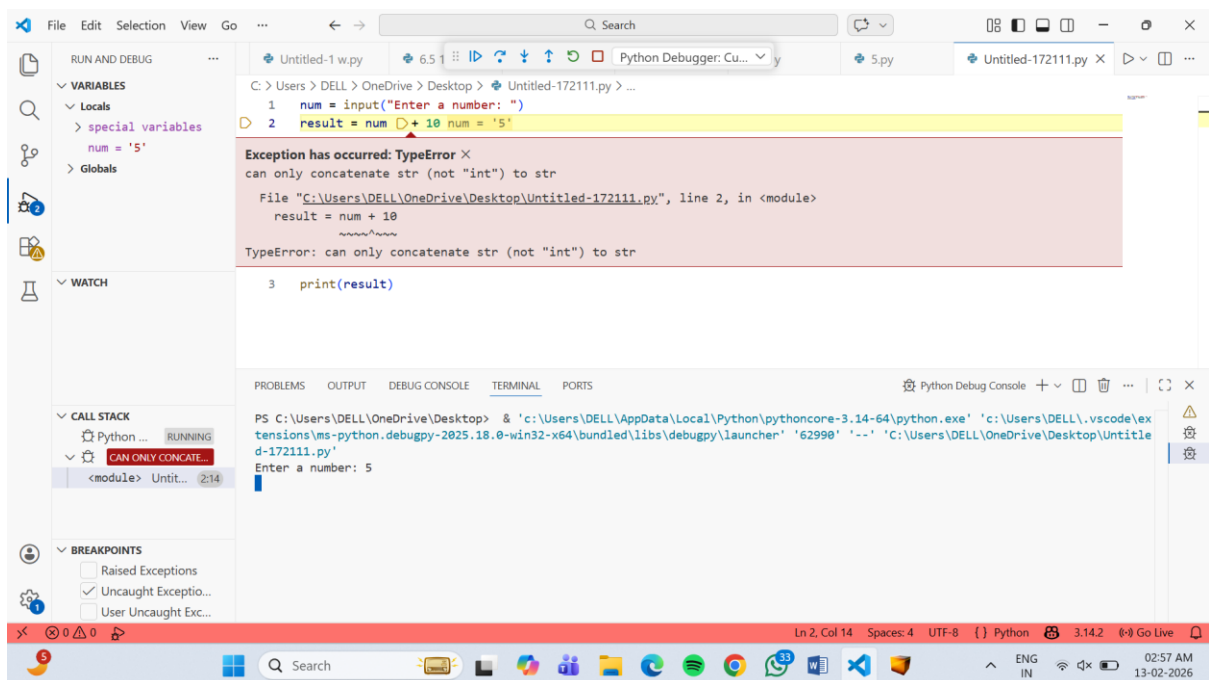
BT.NO: 29

LAB 7: ERROR DEBUGGING WITH AI: SYSTEMATIC APPROACHES TO FINDING AND FIXING BUGS

TASK 1 – RUNTIME ERROR DUE TO INVALID INPUT TYPE

● A Python program accepts user input and performs arithmetic operations. However, the program throws a runtime error because the input is treated as a string instead of a numeric type.

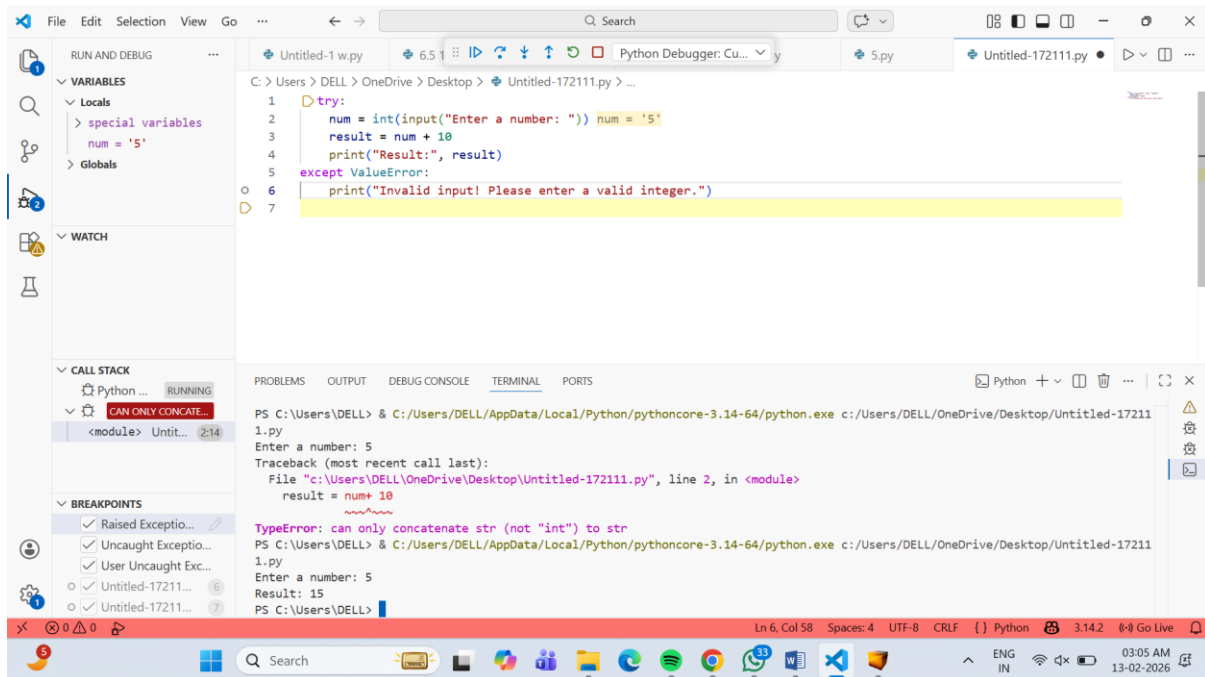
CODE :



```
C:\Users\DELL\OneDrive\Desktop> Python Debugger: Cu... y
1 num = input("Enter a number: ")
2 result = num + 10 num = '5'
3 print(result)

Exception has occurred: TypeError X
can only concatenate str (not "int") to str
File "C:\Users\DELL\OneDrive\Desktop\Untitled-172111.py", line 2, in <module>
    result = num + 10
    ~~~~~
TypeError: can only concatenate str (not "int") to str

PS C:\Users\DELL\OneDrive\Desktop> & 'c:\Users\DELL\AppData\Local\Python\pythoncore-3.14-64\python.exe' 'c:\Users\DELL\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '62990' '--' 'C:\Users\DELL\OneDrive\Desktop\Untitled-172111.py'
Enter a number: 5
```



JUSTIFICATION:

- `int(input())` converts the string input into an integer.
- Arithmetic operations can now be performed safely.
- The runtime error is eliminated.

TASK 2 – INCORRECT FUNCTION RETURN VALUE

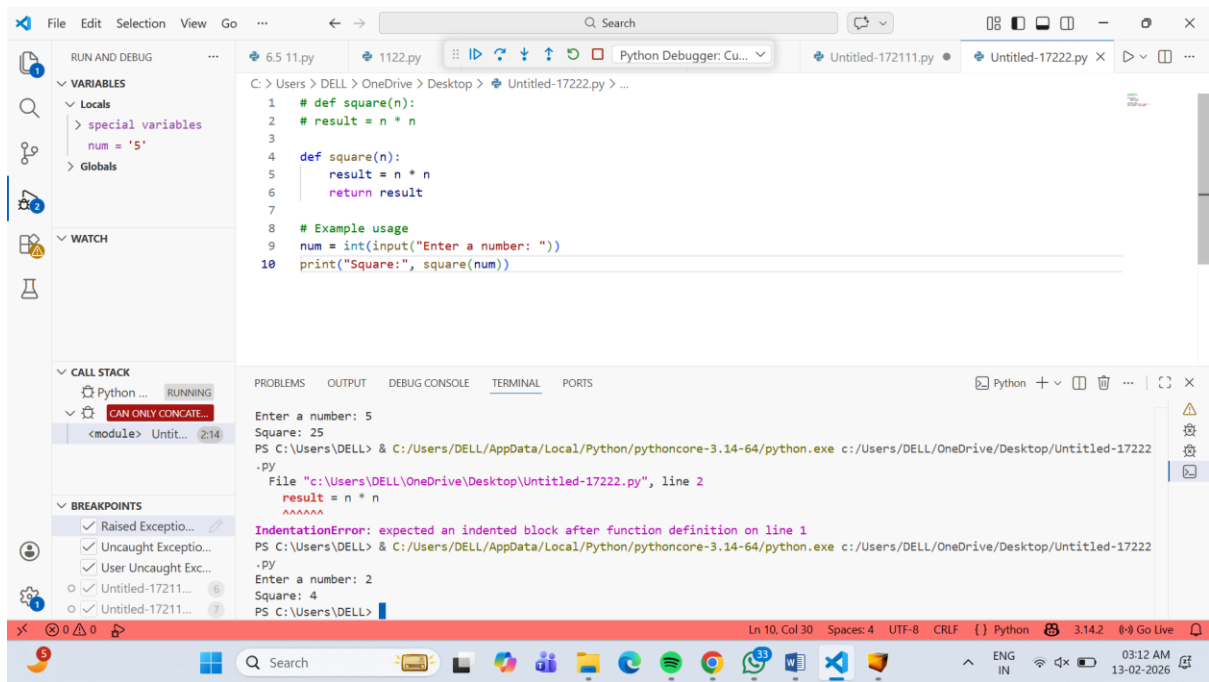
A function is designed to calculate the square of a number, but it does not return the computed result properly.

Example (Buggy Code):

```
def square(n):
```

```
    result = n * n
```

CODE :



JUSTIFICATION:

- $n * n$ correctly computes the square
- `return result` ensures the value is passed back.
- The function now behaves as expected.

TASK 3 – INDEXERROR IN LIST TRAVERSAL

A Python program iterates over a list using incorrect index limits, causing an `IndexError`.

Example (Buggy Code):

```
numbers = [10, 20, 30]
```

```
for i in range(0, len(numbers)+1):
```

```
print(numbers[i])
```

CODE:

The screenshot shows the Visual Studio Code interface with a Python file named 72333.py. The code is as follows:

```
1 numbers = [10, 20, 30]
2 for i in range(0, len(numbers)+1):
3     print(numbers[i])
```

The terminal output shows the command to run the script and the resulting error:

```
PS C:\Users\DELL> & C:/Users/DELL/AppData/Local/Python/pythoncore-3.14-64/python.exe c:/Users/DELL/OneDrive/Desktop/72333.py
Enter a number: 2
Square: 4
File "c:\Users\DELL\OneDrive\Desktop\72333.py", line 3
    print(numbers[i])
    ^^^^^
IndentationError: expected an indented block after 'for' statement on line 2
```

The error message indicates that the code block under the 'for' loop is not properly indented.

The screenshot shows the Visual Studio Code interface with a Python file named 72333.py. The code is as follows:

```
1 #numbers = [10, 20, 30]
2 #for i in range(0, len(numbers)+1):
3 #print(numbers[i])
4
5 numbers = [10, 20, 30]
6 for num in numbers:
7     print(num)
```

The terminal output shows the command to run the script and the resulting errors:

```
PS C:\Users\DELL> & C:/Users/DELL/AppData/Local/Python/pythoncore-3.14-64/python.exe c:/Users/DELL/OneDrive/Desktop/72333.py
Traceback (most recent call last):
  File "c:\Users\DELL\OneDrive\Desktop\72333.py", line 5, in <module>
    for num in numbers:
               ^^^^^^^
NameError: name 'numbers' is not defined. Did you forget to import 'numbers'?
```

The error message indicates that the variable 'numbers' is not defined before it is used in the loop.

JUSTIFICATION:

- range (0, len(numbers)) prevents out-of-range access.
- The enhanced loop avoids index handling entirely.
- The Index Error is fully eliminated.

TASK 4 – UNINITIALIZED VARIABLE USAGE

A program uses a variable in a calculation before assigning it any value.

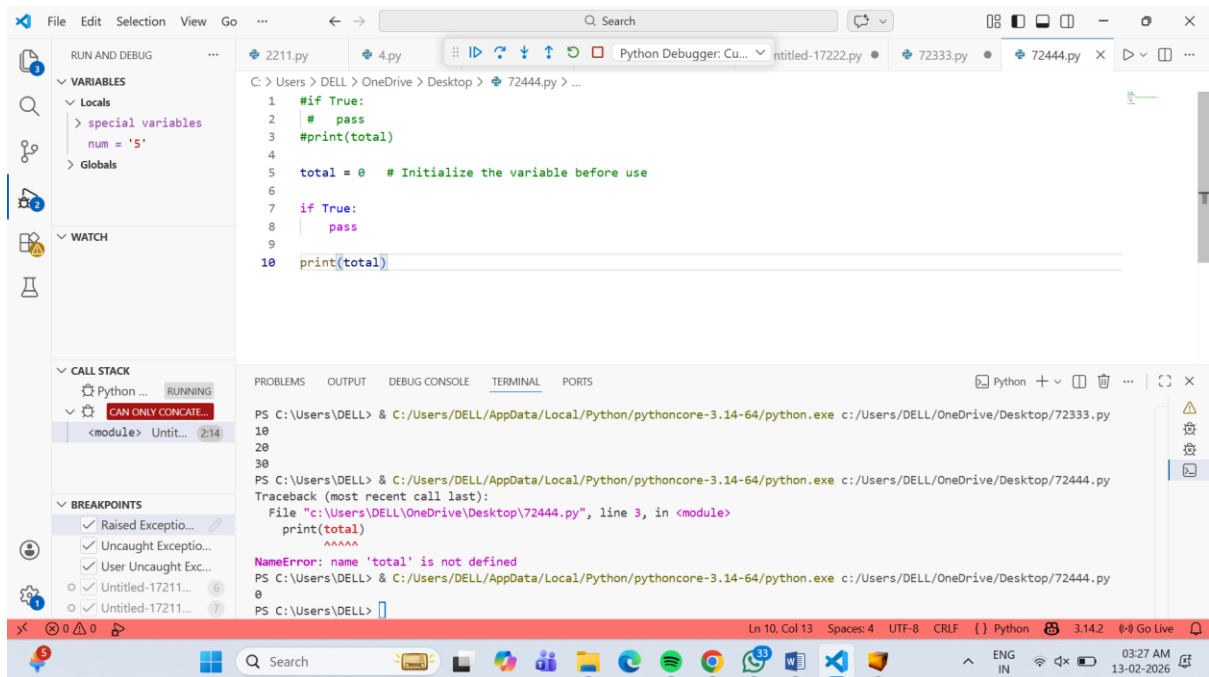
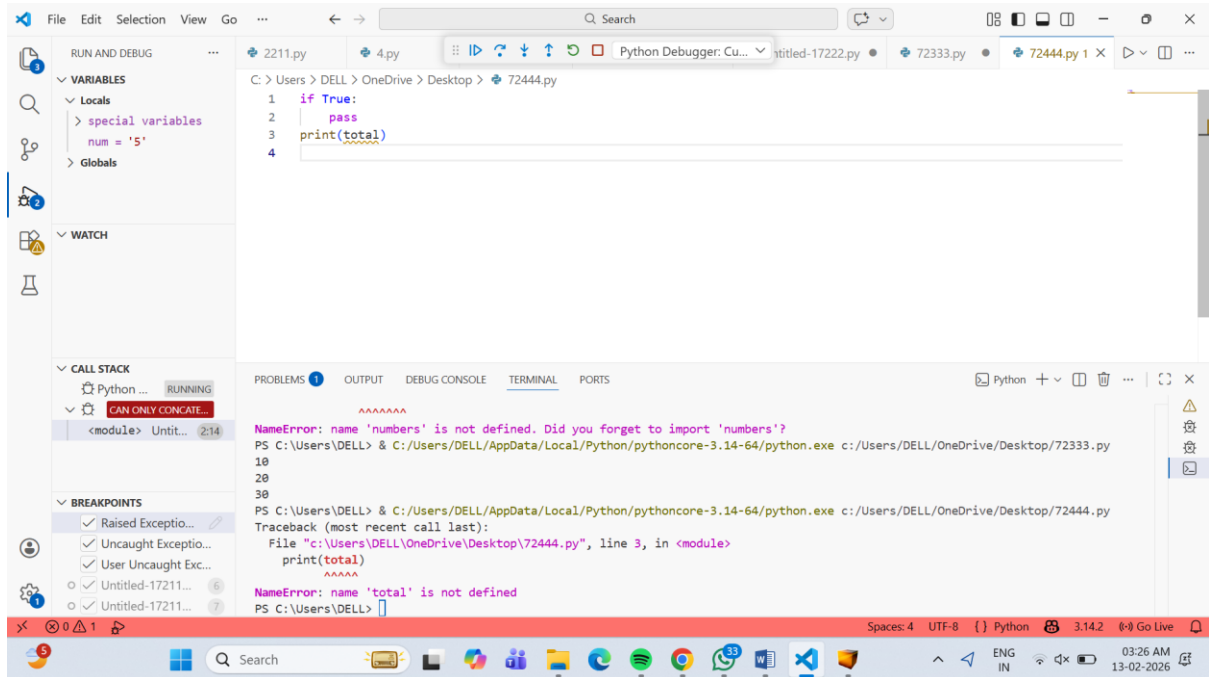
Example (Buggy Code):

if True:

pass

print(total)

CODE :



JUSTIFICATION:

- Initializing total ensures it exists in memory
- The program can now safely access and print the variable.
- The runtime error is eliminated.

TASK 5 – LOGICAL ERROR IN STUDENT GRADING SYSTEM

A grading program assigns incorrect grades due to improper conditional logic.

Example (Buggy Code):

```
marks = 85
```

```
if marks >= 90:
```

```
    grade = "A "
```

```
elif marks >= 80:
```

```
    grade = "C "
```

```
else:
```

```
    grade = "B "
```

```
print(grade)
```

CODE:

The screenshot shows the Visual Studio Code interface with a Python file named 72555.py. The code defines a variable 'marks' with a value of 85 and a conditional statement to assign a grade based on the marks. The code is as follows:

```
1 marks = 85
2
3 if marks >= 90:
4     grade = "A"
5 elif marks >= 80:
6     grade = "B"
7 elif marks >= 70:
8     grade = "C"
9 else:
10    grade = "D"
11
12 print("Grade:", grade)
```

The terminal output shows the execution of the script, which results in a `NameError: name 'total' is not defined`. The error message is displayed in the terminal window, indicating that the variable 'total' has not been defined before it is used in the `print(total)` statement.

The screenshot shows the Visual Studio Code interface with the same Python file named 72555.py. The code is identical to the previous screenshot, but the terminal output now shows a `SyntaxError: unterminated string literal (detected at line 8)`. The error message is displayed in the terminal window, indicating that the string literal in the code is not properly closed.

JUSTIFICATION:

- Initializing total ensures it exists in memory.
- The program can now safely access and print the variable.
- The runtime error is eliminated.